

Laboratorio 11 (A-L)

Collezioni, Generics e Interfacce Funzionali in Java

Michele Gentile

(esercizi preparati dal Prof. Danilo Francati)

Maggio 2026

1 Gestione Canzoni

1. Si definisca una classe **Canzone** i cui oggetti sono costruiti con il nome della canzone e il nome dell'autore.
2. Si crei una classe **Raccolta** che contiene (ovvero i cui oggetti sono costruiti con) un insieme di canzoni.
 - In una prima versione, si utilizzi una struttura **HashSet** per memorizzare l'insieme di canzoni.
 - Si implementino correttamente i metodi **equals** e **hashCode** in modo da evitare duplicati di canzoni sostanzialmente uguali (ovvero con lo stesso nome e autore).
 - La classe *Raccolta* è dotata di un metodo **toString** che restituisce una rappresentazione sotto forma di stringa dell'insieme di canzoni; analogamente, anche la classe *Canzone* è dotata dello stesso metodo.
3. Si implementi quindi la classe **RaccoltaOrdinata** che utilizza **TreeSet** per mantenere un ordine sulle canzoni.
 - L'ordinamento delle canzoni deve essere basato prima sul nome della canzone e, in caso di uguaglianza, sul nome dell'autore.

🔗 Obiettivo: Interfacce Set e Ordinamento

L'obiettivo è comprendere la differenza tra collezioni non ordinate (*HashSet*) e ordinate (*TreeSet*), gestendo l'identità degli oggetti tramite *equals/hashCode* e il confronto tramite *Comparable*.

💡 Tip: Implementazione di compareTo

Per l'ordinamento richiesto, è necessario implementare l'interfaccia **Comparable**. Il metodo **compareTo** deve riflettere la priorità: prima il nome del brano e poi l'autore.

2 La MultiMappa

1. Una **multimappa** è una mappa che ammette più valori a fronte di una data chiave. Si crei una classe *MultiMappa* generica sul tipo delle chiavi (**K**) e dei valori (**V**).
2. La classe deve implementare i seguenti metodi:
 - `put(k, v)`: associa il valore alla chiave specificata.
 - `putAll(MultiMappa)`: aggiunge tutti gli elementi della multimappa in input alla mappa corrente.
 - `removeAll(MultiMappa)`: rimuove tutte le chiavi della multimappa in input dalla mappa corrente.
 - `get(k)`: restituisce l'insieme dei valori associati alla chiave.
 - `get(k, p)`: restituisce solo i valori associati alla chiave k che soddisfano il predicato p .
 - `values()`: restituisce l'elenco (con duplicati) dei valori contenuti nella multimappa.
 - `valueSet()`: restituisce l'insieme dei valori contenuti nella multimappa.
 - `transformToMultiMappa`: restituisce una multimappa in cui le coppie (k, v) sono trasformate in (k, z) secondo una funzione $(k, v) \rightarrow z$.
 - `mapEach`: sostituisce ciascun valore v con un valore dello stesso tipo secondo una funzione $(k, v) \rightarrow v'$.
3. La classe deve essere **iterabile** sulle coppie (k, v) mediante una classe interna *Elemento*.

🔗 Obiettivo: Generics e Interfacce Funzionali

L'esercizio richiede l'uso di tipi generici `<K, V>` e delle interfacce `Predicate` e `BiFunction` per manipolare i dati in modo flessibile.

⚠️ Attenzione: Inizializzazione della Mappa

Ricordate che la struttura interna deve essere `Map<K, HashSet>`. Ogni volta che si inserisce una nuova chiave, è necessario istanziare correttamente il relativo `HashSet` per evitare *NullPointerException*.